

TRANSACTIONS ET GESTION DES ACCÈS CONCURRENTS

➤ DÉFINITION

➤ PRINCIPE DE LA TRANSACTION

➤ COMMANDES COMMIT ET ROLLBACK

➤ DEMARRER UNE TRANSACTION

➤ JALON DE TRANSACTION

➤ ACID

➤ CONFLITS DES TRANSACTIONS

➤ VERROUS

➤ EXEMPLES DE TRANSACTIONS AVEC VERRON

➤ DEADLOCK

➤ RESOLUTION DES DEADLOCK

➤ ANOMALIES LIÉES AUX ACCÈS CONCURRENTS

➤ NIVEAUX D'ISOLATION DE TRANSACTION

Définition

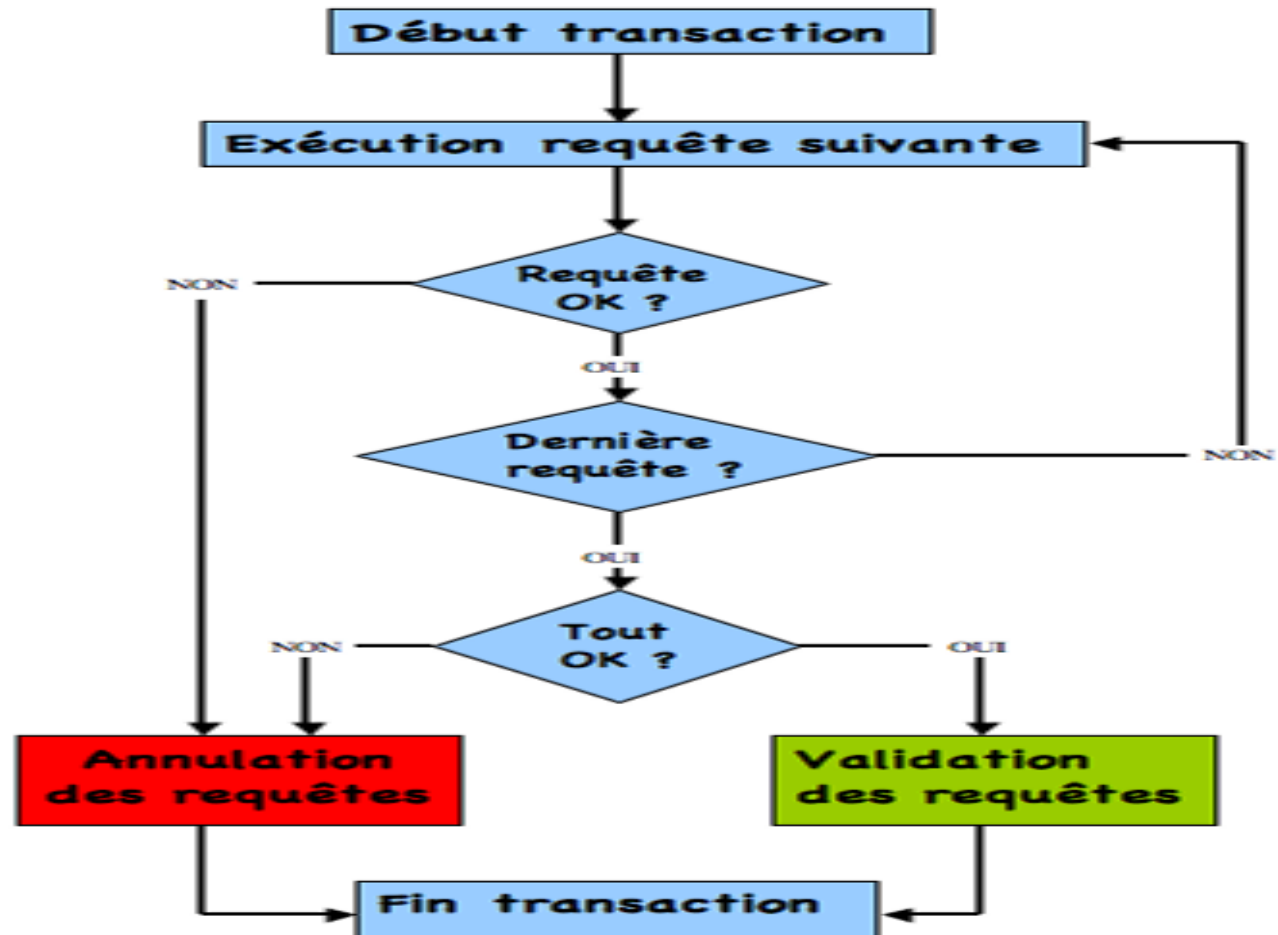
- ❖ **Une Transaction** est une fonctionnalité indispensable, permettant de sécuriser une application utilisant une base de données.
- ❖ **Sans transactions,**
 - Certaines opérations risqueraient d'être réalisées à moitié,
 - La moindre erreur, ou la moindre interruption pourrait avoir des conséquences énormes.

Principe de la Transaction

- ❖ **Une transaction** est donc un **ensemble de requêtes** qui sont **exécutées en un seul bloc**.
- ❖ Ainsi, si une des requêtes du bloc échoue, on peut décider d'annuler tout le bloc de requêtes (ou de quand même valider les requêtes qui ont réussi).

Principe de la Transaction

❖ Schéma de déroulement d'une transaction



Principe de la Transaction

- ❖ Il n'est cependant **pas possible d'utiliser les transactions sur n'importe quelle table.**
- ❖ Pour les supporter, une table doit être **transactionnelle**, ce qui. Avec **MySQL**, ce sont les tables du moteur de stockage **InnoDB** qui sont transactionnelles.

Commandes COMMIT et ROLLBACK

❖ **COMMIT** est une commande qui permet de **valider les changements** apportés par une requête

- Par défaut, MySQL **commite** (valide) toutes ses requêtes. Pour quitter ce mode, il faut exécuter la requête suivante:

✓ **SET autocommit=0;**

❖ **ROLLBACK** est une commande qui permet **d'annuler les changements** apportés par une requête.

- Tant que les modifications apportées par une requête ne sont pas validées, vous pouvez à tout moment les annuler avec la commande **rollback;**

Démarrer une transaction

- ❖ Pour démarrer une transaction, on utilise la commande:
 - **START TRANSACTION;**
- ❖ **Exemple de transaction**
 - Dans un système de gestion de comptes bancaire, on veut transférer 50 000 FCFA du compte A vers le compte B.
 - La structure de la table compte est la suivante:

```
CREATE TABLE comptes (  
    id_compte INT PRIMARY KEY,  
    titulaire VARCHAR(50),  
    solde INT );
```

Démarrer une transaction

❖ Exemple de transaction

START TRANSACTION;

UPDATE comptes

-- 1) Débiter le compte A (id = 1)

SET solde = solde - 50000

WHERE id_compte = 1;

UPDATE comptes

-- 2) Créditer le compte B (id = 2)

SET solde = solde + 50000

WHERE id_compte = 2;

COMMIT;

-- Si une erreur est détectée : -- ROLLBACK;

Jalon de transaction

- ❖ Il n'est pas possible de démarrer une transaction à l'intérieur d'une transaction, on utilise alors les **jalons de transaction**.
- ❖ Un **jalon de transaction** est un **point de repère**, qui permet d'annuler toutes les requêtes exécutées depuis ce jalon, et non toutes les requêtes de la transaction
- ❖ Il permet de conserver des modifications apportées par les certaines requêtes d'une transaction, en cas d'interruption de la transaction

Jalon de transaction

- ❖ Pour créer un jalon
 - **SAVEPOINT** nom_jalon;
- ❖ Annuler les requêtes exécutées depuis le jalon "nom_jalon"
 - **ROLLBACK TO [SAVEPOINT]** nom_jalon;
- ❖ *Exemple d'utilisation de rollback et savepoint*

Jalon de transaction

❖ Exemple d'utilisation de rollback et savepoint

START TRANSACTION;

SAVEPOINT debut_transfert; -- debut_transfert est le nom du jalon

UPDATE comptes

SET solde = solde - 50000

WHERE id_compte = 1;

UPDATE comptes

SET solde = solde + 50000

WHERE id_compte = 2;

COMMIT; -- En cas de problème sur le crédit : ROLLBACK TO debut_transfert;

Jalon de transaction

❖ **Notez-Bien:** Certaines modifications ne peuvent pas être annulées par le ROLLBACK, il s'agit de:

- Les commandes du langage définition des données
✓ Create, Alter, Drop, Rename
- Les commandes de création, de modification et de suppression des utilisateurs
- La création et suppression de verrous de table
- Le chargement de données avec LOAD DATA

ACID

- ❖ **ACID** est un ensemble de critères qu'un système utilisant les transactions doit respecter pour être fiable.
- ❖ Ces critères sont au nombre de quatre :
 - Atomicité,
 - Cohérence,
 - Isolation,
 - Durabilité.

ACID: Atomicité

- ❖ Une transaction doit être **atomique**, c'est-à-dire qu'elle doit former une **entité complète et indivisible**.
- ❖ Chaque élément de la transaction, chaque requête effectuée, ne peut exister que dans la transaction.

ACID: Cohérence

- ❖ Une transaction est **cohérente** lorsque, quelque soit l'issue de la transaction (réussie ou interrompue), les données de la base restent cohérentes.
- ❖ Dans une transaction, **tant que la totalité des étapes n'a pas été réalisée avec succès, les données restent dans leur état initial**

ACID: Isolation

- ❖ Chaque transaction doit être **isolée**, donc **ne pas interagir avec une autre transaction**
- ❖ Des transactions simultanées doivent agir comme si elles étaient seules sur la base.
- ❖ Elles ne voient pas les données **non validées** des autres transactions.

ACID: **D**urabilité

- ❖ Une fois la transaction terminée, les données résultant de cette transaction doivent être stockées de manière **durable**, et pouvoir être récupérées même en cas de crash du serveur par exemple

Conflits de transaction

- ❖ Un **conflit de transaction** se produit lorsque plusieurs transactions accèdent aux mêmes données de manière incompatible
- ❖ Ce qui peut entraîner des incohérences ou des erreurs dans la base de données.
- ❖ Les types de conflits:
 - Les conflits de **lecture-écriture**
 - Les conflits **d'écriture-écriture**

Conflits de transaction

- ❖ Un **Conflit de lecture-écriture** se produit Lorsqu'une transaction essaie de lire des données qui sont en train d'être modifiées par une autre transaction.
 - Par **exemple**, une transaction T1 met à jour un enregistrement, tandis qu'une transaction T2 essaie de lire cet enregistrement avant que T1 n'ait terminé.
- ❖ Un **Conflit d'écriture-écriture** se produit lorsque deux transactions essaient de modifier la même donnée en même temps.
 - ✓ Cela peut entraîner une perte de mise à jour si les modifications ne sont pas correctement gérées.

Gestion de conflits de transaction

- ❖ Les systèmes de gestion de base de données (SGBD) utilisent des **mécanismes de contrôle de concurrence** pour résoudre les conflits de transaction, notamment :
 - Le **verrouillage**
 - Les **transactions en séries**
 - Les **rejets ou rollback**

Gestion de conflits de transaction

- ❖ **Verrouillage (Locking)**: Les SGBD utilisent des **verrous** pour empêcher les conflits en bloquant l'accès concurrent à une donnée spécifique.
- ❖ **Transaction en série (Serializable transactions)** : L'idée est que les SGBD s'assurent que les transactions sont exécutées de manière à ce qu'elles produisent le même résultat qu'une exécution séquentielle des transactions.
- ❖ **Rejet ou rollback** : Lorsque des conflits se produisent et qu'une transaction ne peut pas être validée, elle est annulée, et les modifications effectuées jusqu'à ce point sont **rejetées (rollback)** pour maintenir la cohérence de la base de données.

Verrous

- ❖ Les **verrous** permettent de **sécuriser les requêtes** en bloquant ponctuellement et partiellement l'accès aux données
- ❖ Ils sont indispensables pour la gestion des conflits de transactions
- ❖ Lorsqu'une session MySQL pose un verrou sur un élément de la base de données, cela veut dire qu'il **restreint, voire interdit, l'accès à cet élément aux autres sessions MySQL** qui voudraient y accéder

Verrous

- ❖ Il est possible de poser un verrou sur une table entière, ou seulement sur une ou plusieurs lignes d'une table.
- ❖ Etant donné qu'un verrou empêche l'accès d'autres sessions, il est en général plus intéressant de poser un verrou sur la plus petite partie de la base possible.
- ❖ Cette notion d'accès simultané aux données par plusieurs sessions différentes s'appelle la concurrence.

Verrous sur une ligne

- ❖ Les verrous de ligne peuvent être de deux types :
 - **Les verrous partagés** : permettent aux autres sessions de lire les données, mais pas de les modifier
 - **Les verrous exclusifs** : ne permettent ni la lecture ni la modification des données

Verrous sur une ligne

- ❖ Les **requêtes de modification et suppression des données** posent **automatiquement un verrou exclusif** sur les lignes concernées
 - Il s'agit des lignes sélectionnées par la clause WHERE, ou toutes les lignes s'il n'y a pas de clause WHERE
- ❖ Les **requêtes d'insertion** quant à elles posent un **verrou exclusif sur la ligne insérée**.
- ❖ Les **requêtes de sélection**, par défaut, **ne posent pas de verrous**. Il faut donc en poser explicitement au besoin.

Verrous partagés

- ❖ Pour poser un **verrou partagé**, on utilise **LOCK IN SHARE MODE** à la fin de la requête SELECT.
- ❖ **Ex:**

```
SELECT *  
FROM Commande  
WHERE idprod = 100  
LOCK IN SHARE MODE;
```

 - ✓ Cette requête pose un **verrou partagé** sur les lignes de la table **Commande** pour lesquelles **idprod** vaut 100
 - ✓ **Ce verrou signifie pour les autres sessions :** "Je suis en train de lire ces données. Vous pouvez venir les lire aussi, mais pas les modifier tant que je n'ai pas terminé."

Verrous exclusifs

- ❖ Pour poser un **verrou exclusif**, on utilise **FOR UPDATE** à la fin de la requête SELECT.
- ❖ **Ex:** SELECT *
FROM Commande
WHERE idprod= 100
FOR UPDATE;
- ✓ **Ce verrou signifie aux autres sessions :** "Je suis en train de lire ces données dans le but probable de faire une modification. Ne les lisez ni les modifiez pas avant que j'aie fini "

verrou de ligne

- ❖ Les **verrous de ligne** ne sont donc pas posés par des commandes spécifiques, mais par des requêtes de **sélection**, **insertion** ou **modification**.
- ❖ Ces verrous existent donc uniquement tant que la requête qui les a posés interagit avec les données
- ❖ Par conséquent, **ce type de verrou s'utilise en conjonction avec les transactions** afin de gérer les potentiels **conflits de transaction**
- ❖ En effet, hors transaction, dès qu'une requête est lancée, elle est effectuée et les éventuelles modifications des données sont **immédiatement validées**.

verrou de ligne

- ❖ **Par contre**, dans le cas d'une requête faite dans une transaction, les changements ne sont pas validés tant que la transaction n'a pas été commitée.
- ❖ Un verrou de ligne est donc lié à la transaction dans laquelle il est posé. Dès que l'on fait un COMMIT ou un ROLLBACK de la transaction, le verrou est levé

Exemple de transaction avec verrou

❖ Exemple 1: Verrou posé sur une requête de modification

✓ Session 1

START TRANSACTION;

UPDATE Client SET pays = 'Suisse'

WHERE id = 8; //un verrou exclusif sera posé sur la ligne avec id = 8

✓ Session 2 _Cas1: Transaction sans verrou

START TRANSACTION;

SELECT *

FROM Client

WHERE id = 8; //pas de verrou

Exemple de transaction avec verrou

- ❖ La session 1 a posé un **verrou exclusif automatiquement** en faisant un **UPDATE**.
- ❖ La session 2 fait une simple sélection, sans poser de verrou. Il n'y a pas de problème, **la requête va s'exécuter normalement (pas de conflit)**
 - Parce que lorsqu'une session démarre une transaction, **elle prend en quelque sorte une photo des tables dans leur état actuel** (les modifications non commitées n'étant pas visibles).
 - La transaction va alors travailler sur la base de cette photo, tant qu'on ne lui demande pas d'aller vérifier que les données n'ont pas changé.
 - Donc le SELECT ne voit pas les changements, et ne se heurte pas au verrou, puisque celui-ci est **posé sur les lignes de la table, et non pas sur la photo de cette table que détient la session 2**

Exemple de transaction avec verrou

❖ Exemple 1: Verrou posé par une requête de modification

✓ Session 1 (requête inchangée)

START TRANSACTION;

UPDATE Client SET pays = 'Suisse'

WHERE id = 8; //un verrou exclusif sera posé sur la ligne avec id = 8

✓ Session 2 _Cas2: Transaction avec verrou

START TRANSACTION;

SELECT *

FROM Client

WHERE id = 8

LOCK IN SHARE MODE; //on essaye de poser un verrou partagé

Exemple de transaction avec verrou

- ❖ La session 1 a posé un **verrou exclusif automatiquement** en faisant un **UPDATE**.
- ❖ La session 2 fait une requête de sélection, en essayant de poser un verrou partagé. **Mais elle ne va pas s'exécuter**
 - Lorsqu'une session pose un verrou sur une table, elle est obligée de travailler avec la table, et pas seulement sur sa photo.
 - La session 2 ira alors d'abord chercher les lignes les plus à jour. Elle tombera sur le verrou posé par la session 1, elle sera par conséquent bloquée jusqu'à ce que la session 1 ôte le verrou exclusif.

Exemple de transaction avec verrou

❖ Exemple 2: Verrou posé par une requête d'insertion

✓ Session 1 //un verrou exclusif posé par l'insertion

```
START TRANSACTION;  
INSERT INTO Adoption (client_id, animal_id, date_reservation, prix)  
VALUES (12, 75, NOW(), 10.00);
```

✓ Session 2 _Cas1

```
START TRANSACTION;  
SELECT *  
FROM Adoption  
WHERE client_id > 13  
LOCK IN SHARE MODE;
```

✓ Session 2 _Cas2

```
START TRANSACTION;  
SELECT *  
FROM Adoption  
WHERE client_id < 13  
LOCK IN SHARE MODE;
```

Exemple de transaction avec verrou

- ❖ La **session 1** insère une adoption pour le **client 12** et pose un verrou exclusif sur cette ligne.
- ❖ La **session 2** fait deux requêtes SELECT en posant un verrou partagé :
 - l'une qui sélectionne les adoptions des clients avec **un id supérieur à 13**;
 - l'autre qui sélectionne les adoptions des clients avec **un id inférieur à 13**.
- ❖ Seule la seconde requête SELECT se heurte au verrou posé par la première session, puisqu'elle tente de récupérer notamment les adoptions du **client 12**, dont une est verrouillée.

Exemple de transaction avec verrou

❖ Exemple 3: Verrou posé par une requête de sélection

✓ Session 1 // avec verrou partagé

```
START TRANSACTION;  
SELECT *  
FROM Client  
WHERE id < 5  
LOCK IN SHARE MODE;
```

✓ Session 2 _Cas1 // sans verrou

```
START TRANSACTION;  
SELECT *  
FROM Client  
WHERE id BETWEEN 3 AND 8;
```

- ❖ La session 1 pose un verrou partagé sur les clients 1, 2, 3 et 4.
- ❖ La session 2 fait une requête de sélection sans verrou, donc travaille sur une "photo" de la table et pas sur les vraies données, donc elle s'exécutera sans souci.

Exemple de transaction avec verrou

❖ Exemple 3: Verrou posé par une requête de sélection

✓ Session 1 // avec verrou partagé ✓ Session 2 _Cas2 // avec verrou partagé

```
START TRANSACTION;  
SELECT *  
FROM Client  
WHERE id < 5  
LOCK IN SHARE MODE;
```

```
START TRANSACTION;  
SELECT *  
FROM Client  
WHERE id BETWEEN 3 AND 8  
LOCK IN SHARE MODE;
```

❖ La session 1 pose un verrou partagé sur les clients 1, 2, 3 et 4.

❖ La session 2 fait une requête de sélection avec verrou partagée sur des lignes concernées par un verrou partagé, alors elle s'exécutera sans souci.

Exemple de transaction avec verrou

❖ Exemple 3: Verrou posé par une requête de sélection

✓ Session 1 // avec verrou partagé ✓ Session 2 _Cas3 // avec verrou partagé

```
START TRANSACTION;  
SELECT *  
FROM Client  
WHERE id < 5  
LOCK IN SHARE MODE;
```

```
START TRANSACTION;  
SELECT *  
FROM Client  
WHERE id BETWEEN 3 AND 8  
LOCK IN SHARE MODE;
```

❖ La session 1 pose un verrou partagé sur les clients 1, 2, 3 et 4.

❖ La session 2 fait une requête de sélection en essayant de poser un verrou partagée sur des lignes concernées par un verrou partagé, donc elle s'exécutera sans souci.

Exemple de transaction avec verrou

❖ Exemple 3: Verrou posé par une requête de sélection

✓ Session 1 // avec verrou partagé ✓ Session 2 _Cas3 // avec verrou exclusif

START TRANSACTION;

SELECT *

FROM Client

WHERE id < 5

LOCK IN SHARE MODE;

START TRANSACTION;

SELECT *

FROM Client

WHERE id BETWEEN 3 AND 8

FOR UPDATE;

❖ La session 1 pose un verrou partagé sur les clients 1, 2, 3 et 4.

❖ La session 2 fait une requête de sélection en essayant de poser un verrou exclusif sur des lignes concernées par un verrou partagé, donc elle sera bloquée. C'est valable pour toute requête UPDATE ou DELETE

Exemple de transaction avec verrou

❖ Remarques

- On pose un **verrou partagé** lorsqu'on fait une requête dans le but de **lire des données**.
- On pose un **verrou exclusif** lorsqu'on fait une requête dans le but immédiat ou non de **modifier des données**.
- Un **verrou partagé sur les lignes x** va permettre aux autres sessions d'obtenir également un verrou partagé sur les lignes x, mais pas d'obtenir un verrou exclusif.
- Un **verrou exclusif sur les lignes x** va empêcher les autres sessions d'obtenir un verrou sur les lignes x, **qu'il soit partagé ou exclusif**.

Deadlock

- ❖ Bien que les **verrous** constituent le principal mécanisme de contrôle de l'accès concurrent aux données,
- ❖ Ils présentent des **limites importantes** lorsqu'ils sont utilisés dans des environnements **multi-utilisateurs**, notamment:
 - Les ralentissement dans l'exécution de certaines tâches
 - Les situation d'attente prolongées
 - Dans certains cas, deux transactions peuvent même **se bloquer mutuellement**, il s'agit de l'interblocage ou **deadlock**

Deadlock

- ❖ Les **deadlocks** (ou **interblocages**) dans les bases de données se produisent lorsqu'un ensemble de transactions se bloque mutuellement de manière circulaire,
- ❖ Ce qui empêche chacune de ces transactions de progresser et d'accomplir ses opérations.
- ❖ Autrement dit, chaque transaction attend qu'une autre libère une ressource qu'elle possède, et au final aucune transaction ne peut avancer.

Deadlock

❖ Comment un deadlock se produit-il ?

- Dans un SGBD, les transactions accèdent souvent à des ressources partagées, comme des lignes de données ou des verrous.
- Un **deadlock** se produit généralement dans les situations suivantes :
 - ✓ Transaction A verrouille Ressource 1.
 - ✓ Transaction B verrouille Ressource 2.
 - ✓ Transaction A attend que Ressource 2 soit libérée par Transaction B.
 - ✓ Transaction B attend que Ressource 1 soit libérée par Transaction A.

Deadlock

- ❖ Aucun des deux processus ne peut continuer, car chacun attend que l'autre termine une action pour libérer la ressource dont il a besoin.
- ❖ Cela crée un **blocage circulaire** sans fin, d'où le nom **deadlock** (littéralement "blocage mortel").

Résolution des Deadlocks

❖ Les **SGBD modernes** incluent des mécanismes pour **détecter et résoudre les deadlocks**. Voici quelques stratégies utilisées pour résoudre un deadlock :

- **Détection automatique**
- **Prévention** des deadlocks
- **Gestion par priorité**

Résolution des Deadlocks

❖ Détection automatique

- Le SGBD détecte périodiquement si un deadlock existe.
- Si un tel blocage est détecté, il choisit généralement une transaction parmi celles impliquées dans le deadlock pour la terminer (rollback), annulant ainsi son travail et permettant aux autres transactions de continuer.

Résolution des Deadlocks

❖ **Prévention des deadlocks**

■ **Ordre de verrouillage strict**

- ✓ Imposer un ordre de verrouillage des ressources.
- ✓ Si toutes les transactions doivent verrouiller les ressources dans un ordre prédéfini, il est plus difficile que des deadlocks se produisent.

■ **Verrouillage avec délai d'expiration (timeout)**

- ✓ Fixer un délai d'attente pour l'acquisition d'un verrou.
- ✓ Si une transaction ne parvient pas à obtenir un verrou dans ce délai, elle abandonne ou réessaie, ce qui peut éviter un deadlock.

Résolution des Deadlocks

❖ Prévention des deadlocks

■ Utilisation de verrous plus fins

- ✓ Au lieu de verrouiller de grandes portions de données (par exemple, des tables entières),
- ✓ les transactions peuvent verrouiller des unités plus petites (par exemple, des lignes spécifiques), ce qui réduit le risque de deadlock.

Résolution des Deadlocks

❖ Prévention des deadlocks

■ Gestion par priorité

- ✓ Certaines bases de données peuvent attribuer des priorités aux transactions.
- ✓ Si un deadlock est détecté, la transaction ayant la **priorité la plus basse** peut être annulée pour libérer les ressources et permettre aux autres transactions de continuer.

Anomalies liées aux accès concurrents

- ❖ Un **accès concurrent** désigne la situation où plusieurs utilisateurs (ou transactions) **accèdent en même temps aux mêmes données dans la base**.
- ❖ Cela peut être pour lire, modifier ou supprimer des informations
- ❖ Ces accès concurrents peuvent engendrer plusieurs anomalies:
 - **Dirty Read** ou lecture sale
 - **Non Repeatable Read** ou lecture non reproductible
 - **Phantom Read** ou lecture fantôme

Anomalies liées aux accès concurrents

- ❖ Il existe plusieurs anomalies liées aux accès concurrents:
 - **Dirty Read (lecture sale)**: Il s'agit de la lecture d'une donnée non encore validée (non commit)
 - **Non Repeatable Read (lecture non reproductible)**: c'est lorsqu'une valeur relue change parce qu'une autre transaction l'a modifiée
 - **Phantom Read (lecture fantôme)**: c'est lorsque le nombre de lignes change entre deux lectures (insert / delete)
- ❖ Pour réduire les risques d'anomalies liés aux accès concurrents, on va définir les **niveaux d'isolation**

Anomalies liées aux accès concurrents

- ❖ Il existe plusieurs anomalies liées aux accès concurrents:
 - **Dirty Read (lecture sale)**: Il s'agit de la lecture d'une donnée non encore validée (**non commit**)
 - **Non Repeatable Read (lecture non reproductible)**: c'est lorsqu'une valeur relue change parce qu'une autre transaction l'a modifiée
 - **Phantom Read (lecture fantôme)**: c'est lorsque le nombre de lignes change entre deux lectures (dû à un insert / delete)
- ❖ Pour réduire les risques d'anomalies liés aux accès concurrents, on va définir les **niveaux d'isolation**

Niveau d'isolation de transaction

- ❖ Le **niveau d'isolation** d'une transaction est un **paramètre** qui détermine **le degré de visibilité des modifications** faites par une transaction par rapport aux autres transactions concurrentes dans une base de données.
- ❖ Il définit à **quel point une transaction est isolée** des autres pour **éviter les incohérences** lorsque plusieurs transactions s'exécutent en même temps.
- ❖ En résumé, les **niveaux d'isolations** servent à **contrôler la visibilité des données** entre transactions

Niveau d'isolation de transaction

❖ Syntaxe:

- `SET [GLOBAL | SESSION] TRANSACTION ISOLATION LEVEL { READ UNCOMMITTED | READ COMMITTED | REPEATABLE READ | SERIALIZABLE }`
 - ✓ Le mot-clé **GLOBAL** définit le niveau d'isolation pour toutes les sessions qui seront créées dans le futur. Les sessions existantes ne sont pas affectées.
 - ✓ **SESSION** définit le niveau d'isolation pour la session courante.
 - ✓ Si l'on ne précise **ni GLOBAL, ni SESSION**, le niveau d'isolation défini ne concernera que la prochaine transaction que l'on ouvrira dans la **session courante**.

Niveau d'isolation

REPEATABLE READ

-
- ❖ **REPEATABLE READ** → Garantit des lectures stables (les lignes ne changent pas).
 - ✓ Il s'agit du **niveau d'isolation par défaut**
 - ✓ **Repeatable read** signifie "**lecture répétable**", c'est-à-dire que si l'on fait plusieurs requêtes de sélection (non-verrouillantes) de suite, elles donneront toujours le même résultat, quels que soient les changements effectués par d'autres sessions.
 - ✓ Si l'on pense à bien utiliser les verrous là où c'est nécessaire, **c'est un niveau d'isolation tout à fait suffisant**

Niveau d'isolation

REPEATABLE READ

❖ **Exemple:** On suppose que le compte d'id=1 a pour solde **100000**

❖ **Dans chaque session on fait:**

- SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

❖ **Session 1**

START TRANSACTION;

SELECT solde

FROM comptes

WHERE id_compte = 1; -- Résultat : 100000

Session 2

START TRANSACTION;

UPDATE comptes

SET solde = 70000

WHERE id_compte = 1;

COMMIT; -- met à jour le solde à 70000 et valide

- SELECT solde FROM comptes WHERE id_compte = 1;

Niveau d'isolation de transaction

❖ Il existe 4 niveaux d'isolation:

- **READ UNCOMMITTED** → Très rapide mais **risque d'incohérences**.
 - ✓ Le niveau READ UNCOMMITTED fonctionne comme READ COMMITTED, c'est juste qu'il **autorise la "lecture sale"**.
 - ✓ C'est-à-dire qu'une session sera capable de lire des changements encore non commités par d'autres sessions.

Niveau d'isolation de transaction

❖ Exemple d'isolation READ UNCOMMITTED

■ Session 1

```
START TRANSACTION;  
UPDATE Race  
SET prix = 5000  
WHERE id = 7;
```

Session 2

```
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;  
START TRANSACTION;  
SELECT id, nom, espece_id, prix  
FROM Race;
```

- ✓ La **modification faite par la session 1 n'a pas été commitée**. Elle ne sera donc potentiellement jamais validée, auquel cas, elle n'affectera jamais les données.
- ✓ Pourtant, la session 2 voit ce changement de données non-commitées ("Lecture sale"). Bien entendu, il faut éviter de travailler avec ce niveau d'isolation.

Niveau d'isolation de transaction

❖ Il existe 4 niveaux d'isolation:

- **SERIALIZABLE** → Plus sûr mais **plus lent** (transactions comme en file d'attente).
 - ✓ Ce niveau d'isolation se comporte comme REPEATABLE READ sauf que le mode auto-commit est désactivé
 - ✓ Tous les SELECT simples sont implicitement convertis en SELECT ... LOCK INSHARE MODE.